

GraphQL på Datafordeleren

Morten Fuglsang, Septima

Hvorfor dette Oplæg ?



Nuværende Datafordeler

- ÷ Nuværende REST-tjenester med forskellig logik og struktur fra register til register
- ÷ Komplekst for registre at specificere REST-tjenester via DLS – og høj time-to-market for implementering heraf
- ÷ Meget manuelt arbejde ifm. implementering af nye og ændrede REST-tjenester
- ÷ Mange data, der ellers er på Datafordeleren, er ikke udstillet via REST-tjenester
- ÷ Udstillingens output skemaer følger ikke altid grunddatamodellen



Moderniserede Datafordeler

- + GraphQL-udstilling for alle entiteter, som anvendere har adgang til
- + Samme logik og struktur for opslag af data, uanset register
- + Ingen register-specifikation (DLS) af udstillings-skemaer, tjenesteparametre eller tjenestelogik
- + Autogenerering af implementering af nye og ændrede GraphQL-tjenester, med minimalt manuelt arbejde
- + Anvendere kan nøjes med at hente de data der er brug for
- ! Anvendere skal (som hidtil) forholde til sammenhæng mellem data ud fra grunddatamodellen

Indhold

- Introduktion til GraphQL
- GraphQL på datafordeleren
 - Simple opslag
 - Fleksible opslag

Mix af slides og eksempler - stil gerne spørgsmål undervejs...

Hvorfor GraphQL

REST er langt hen ad vejen den dominerende API type

Udfordringer med REST:

- Overfetching og underfetching af data
- Mange API-kald for at hente relaterede data
- Vanskeligt at tilpasse data til forskellige klienter (web, mobil)

Fødselen af GraphQL hos Facebook

- Skabt af Facebook i 2012 som intern løsning
- Bruges først i deres mobile app til nyhedsfeedet
- Offentliggjort som open source i 2015
- Formål: Effektiv og fleksibel datatilgængelighed med ét enkelt kald

GraphQL i dag

- Bred adoption hos store virksomheder (GitHub, Shopify, Twitter)
- Bruges til både frontend og backend integrationer
- Aktivt community og voksende økosystem (Apollo, Relay, Hasura osv.)

Et paradigmeskifte: fra ressourcer til grafer

Hvad er en graf ?

En graf består af:

- **Noder (nodes):** Objekter som fx `Bruger` , `Adresse` , `Postnummer` , `bygning`
- **Kanter (edges):** Relationer mellem objekterne (fx en bruger har en adresse, der ligger i en bygning osv.)
- **Felter (fields):** Informationer på hver node

En GraphQL-query følger forbindelserne i grafen

Du kan spørge på tværs af datarelationer – fleksibelt

Hvad er en query ?

Query: Nøgleordet for en læseoperation

Vi beder om et user-objekt med id =
"123"

Vi specificerer præcis hvilke felter vi vil
have: id, name, email

```
query {  
  user(id: "123") {  
    id  
    name  
    email  
  }  
}
```


Hvordan sender man en forespørgsel ?

GraphQL bruger som regel HTTP POST (kan også være GET)

Body'en indeholder query som tekst ('Næsten' JSON)

```
POST /graphql
```

```
Content-Type: application/json
```

```
{  
  "query": "query { user(id: \"123\") { name email } }"  
}
```

Applikationer

Man kan i praksis godt bygge en **GET** url, men der skal url-encodes en masse ting.

I praksis vil man ofte benytte en applikation der kan lave **POST** forespørgsler:

- Postman / Insomnia / Altair
- Apollo Client
- curl
- GraphQL Playground / GraphiQL

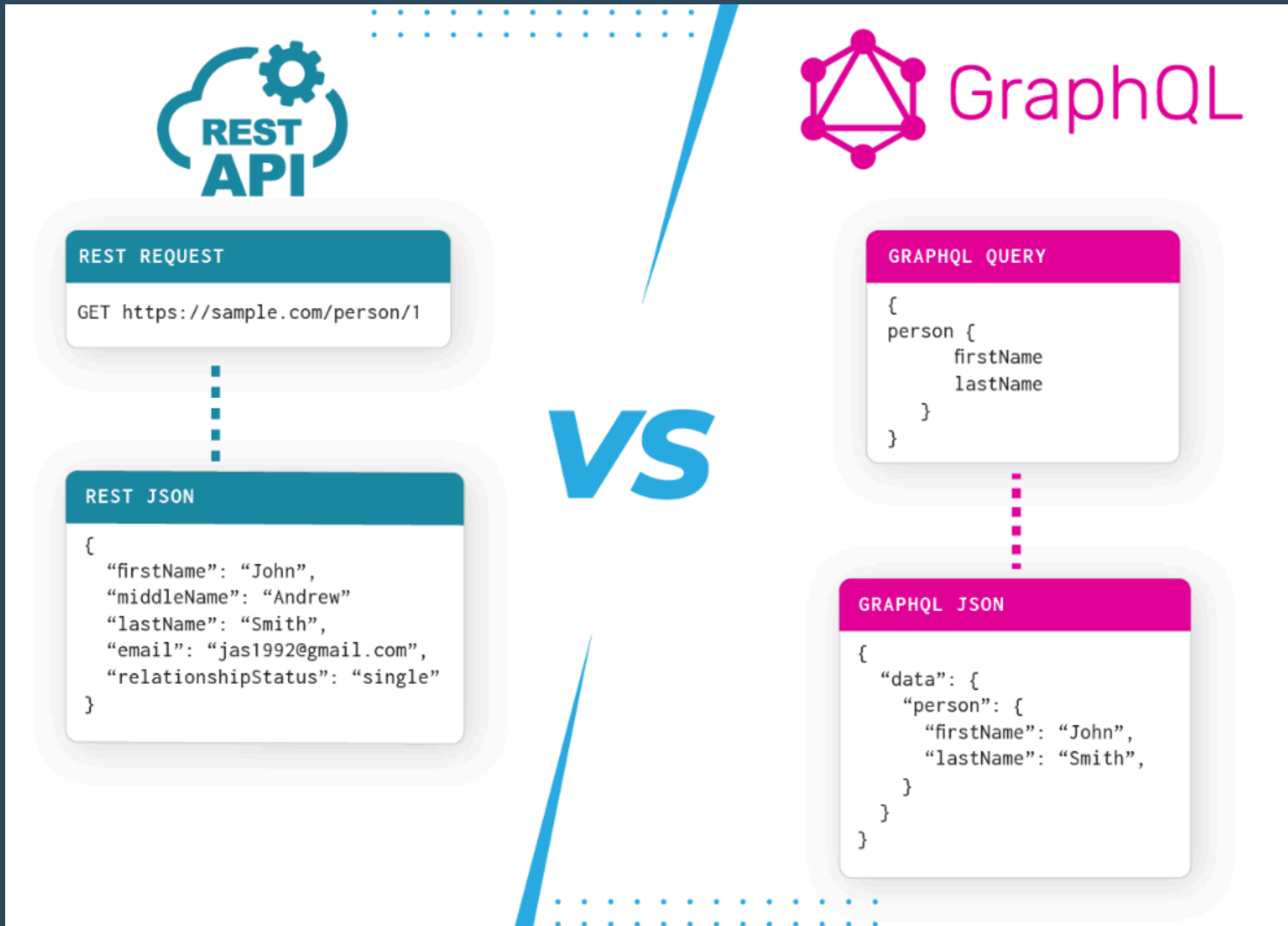
Response

Response indeholder kun de felter der blev forespurgt

Ingen overflødig data – ingen status, links, osv.

Altid under data-nøglen (fejl kommer i errors)

```
{  
  "data": {  
    "user": {  
      "name": "Alice",  
      "email": "alice@example.com"  
    }  
  }  
}
```



GraphQL schemas

Et GraphQL-schema definerer de typer og felter, du kan spørge om i API'et.

Schemaet fungerer som:

- En **kontrakt** mellem klient og server
- En **struktur** over alle data og deres relationer
- En **validering** af dine queries

Et schema består af:

- `type` – definerer objekttyper (f.eks. `Adresse` , `Vejstykke`)
- `Query` – definerer tilgængelige læseoperationer

Det hele er stærkt typet og dokumenteret...

Eksempel fra Datafordeleren

```
query {  
  BBR_Bygning( # Entitet  
    where: { # Standardfiltre  
      kommunekode: { eq: "0550" }  
      virkningsaktoer: { startsWith: "Konvertering" }  
    }  
    virkningstid: "2024-11-12T14:41:33Z" # Bitemporalt filter  
    first: 10 # Paging  
  ) {  
    pageInfo { # Paginginformation  
      startCursor  
      endCursor  
      hasNextPage  
      hasPreviousPage  
    }  
    nodes {  
      id_lokalId  
      kommunekode  
      virkningsaktoer  
    }  
  }  
}
```

- **Entitet** : Hver query skal specificere hvilken entitet den omhandler. Eksemplet ovenfor viser en query for entiteten BBR_Bygning fra BBR og specifikt kun for 3 kolonner (angivet i "nodes"): id_lokalId, kommunekode og virkningsaktoer.
- **Endepunkt**: En query sendes til et GraphQL-endepunkt som et GET- eller POSTrequest.
- **Standardfiltre**: En query kan indeholde standardfiltre der filtrerer data. Query'en ovenfor indeholder to standardfiltre: et lighedsfilter (eq: "0550") på kommunekode-kolonnen og et tekstfilter (startsWith: "Konvertering") på virkningsaktoer-kolonnen.

- **Geometriske filtre:** En query kan indeholde geometriske filtre, der filtrerer data på baggrund af geometri.
- **Bitemporale filtre:** En query kan også indeholde bitemporale filtre. Query'en oven for indeholder et bitemporalt point-in-time-filter (virkningstid: "2024-11-12T14:41:33Z") på virkningstidspunktet.
- **Paging :** En query kan også definere hvordan data pagineres. Query'en oven for returnerer de første 10 rækker (first: 10) givet filtreringen samt paginginformation for resultatsættet (angivet i "pageInfo").

Udviklingsleverance 2 - Det der er i drift nu...

Hvordan får man adgang ?

- Det er nødvendigt at etablere en ny type adgang - det er ikke længere nok med en webbruger og en tjenestebruger.
- Hvis man skal have adgang til frie data, er en token nok, men den skal oprettes

Administration

Log på Datafordelerens Administration og opret de IT-systemer, der skal have adgang til at hente data via Fildownload (HTTPS) og GraphQL.

DATAFORDELER ADMINISTRATION

Bemærk at hvert testmiljø har egen Administration.

TEST 03

TEST 04

TEST 06

Bemærk: Du skal ansøge i Administration for at få adgang til beskyttede data. Under hvert IT-system kan du oprette API-keys eller OAuth Shared Secrets, som du skal indsætte i URL for at hente data via HTTPS. Du kan også oprette og administrere andre anvendere, der skal have adgang til IT-systemer eller Administration.

Velkommen til Datafordeler Administrationen

For at få adgang til data fra Datafordeleren skal du oprette dig som anvender med enten e-mail eller MitID. Derefter kan du logge ind i Datafordelerens Administration og oprette de IT-systemer, der skal have adgang til at hente data. Under hvert IT-system kan du oprette API-keys eller OAuth Shared Secrets, som du skal indsætte i URL for at hente data via HTTPS. Du kan også oprette og administrere andre anvendere, der skal have adgang til IT-systemer eller Administration. Du skal ansøge i Administration for at få adgang til beskyttede data. Bemærk at hvert testmiljø også har egen Administration.

Lad os hente nogle schemaer

`graphql.datafordeler.dk/DAR/v1/schema?apiKey=API-KEY`

`graphql.datafordeler.dk/BBR/v1/schema?apiKey=API-KEY`

`graphql.datafordeler.dk/MAT/v1/schema?apiKey=API-KEY`

Vi starter med at kigge nærmere på DAR schemaet...

Elementer

- Metadata
- Query types
- Filter inputs
- Spatiale filtre

```
schema {  
  query: Query  
}  
  
interface SpatialInterfaceType {  
  "The OGC geometry type of the geometry."  
  type: SpatialGeometryNameEnumType!  
  "The coordinate reference system of the geometry."  
  crs: Int!  
  "The dimensions used by the geometry."  
  dimension: SpatialDimensionEnumType!  
  "The WKT representation of the geometry"  
  wkt: String  
}  
  
""  
Grunddatamodel info:  
name: Adresse  
comment (da): <memo>#NOTES#en sammensat betegnelse, som udpeger o  
definition (da): <memo>#NOTES#en struktureret betegnelse som ang  
example (da): <memo>#NOTES#adressebetegnelse: Gimles Allé 14, 230  
historikmodel: bitemporalitet#NOTES#Values: registreringshistorik  
legalSource: https://www.retsinformation.dk/eli/lta/2018/271  
prefLabel (da): Adresse  
URI: https://data.gov.dk/model/profile/dar/Adresse  
EAID: EAID_6043F4D9_C4CA_44d0_BE88_027166A8B008  
""  
  
type DAR_Adresse {  
  ""  
  Grunddatamodel info:  
  type: CharacterString  
  multiplicity: 1  
  URI: https://data.gov.dk/model/profile/dar/adressebetegnelse
```

Query types

```
"Query type for DAR - Danmarks Adresseregister"
```

```
type Query {
```

```
  DAR_Adressepunkt("Returns the first _n_ elements from the list." first: Int "Returns the elements in the list that
```

```
  DAR_Adresse("Returns the first _n_ elements from the list." first: Int "Returns the elements in the list that cor
```

```
  DAR_Husnummer("Returns the first _n_ elements from the list." first: Int "Returns the elements in the list that c
```

```
  DAR_NavngivenVejKommunedel("Returns the first _n_ elements from the list." first: Int "Returns the elements in th
```

```
  DAR_NavngivenVejPostnummer("Returns the first _n_ elements from the list." first: Int "Returns the elements in th
```

```
  DAR_NavngivenVejSupplerendeBynavn("Returns the first _n_ elements from the list." first: Int "Returns the element
```

```
  DAR_NavngivenVej("Returns the first _n_ elements from the list." first: Int "Returns the elements in the list tha
```

```
  DAR_Postnummer("Returns the first _n_ elements from the list." first: Int "Returns the elements in the list that
```

```
  DAR_SupplerendeBynavn("Returns the first _n_ elements from the list." first: Int "Returns the elements in the li
```

```
}
```

Filter inputs

```
input DAR_AdresseFilterInput {  
  and: [DAR_AdresseFilterInput!]  
  adressebetegnelse: DafSearchableStringOperationFilterInput  
  bygning: DafStringOperationFilterInput  
  datafordelerOpdateringstid: DafDateTimeOperationFilterInput  
  doerbetegnelse: DafStringOperationFilterInput  
  doerpunkt: DafStringOperationFilterInput  
  etagebetegnelse: DafStringOperationFilterInput  
  husnummer: DafStringOperationFilterInput  
  id_lokalId: DafStringOperationFilterInput  
  registreringFra: DafDateTimeOperationFilterInput  
  registreringsaktoer: DafStringOperationFilterInput  
  registreringTil: DafDateTimeOperationFilterInput  
  status: DafStringOperationFilterInput  
  virkningFra: DafDateTimeOperationFilterInput  
  virkningsaktoer: DafStringOperationFilterInput  
  virkningTil: DafDateTimeOperationFilterInput  
}
```


Spatial Filters

```
input SpatialFilterInput {  
  and: [SpatialFilterInput!]  
  or: [SpatialFilterInput!]  
  contains: SpatialContainsOperationFilterInput @cost(weight: "5000")  
  covers: SpatialCoversOperationFilterInput @cost(weight: "5000")  
  crosses: SpatialCrossesOperationFilterInput @cost(weight: "5000")  
  intersects: SpatialIntersectsOperationFilterInput @cost(weight: "5000")  
  within: SpatialWithinOperationFilterInput @cost(weight: "5000")  
  overlaps: SpatialOverlapsOperationFilterInput @cost(weight: "5000")  
  touches: SpatialTouchesOperationFilterInput @cost(weight: "5000")  
  coveredBy: SpatialCoveredByOperationFilterInput @cost(weight: "5000")  
}
```

Metadata

```
type DAR_Adresse {  
    ""  
    Grunddatamodel info:  
    type: CharacterString  
    multiplicity: 1  
    URI: https://data.gov.dk/model/profile/dar/adressebetegnelse  
    prefLabel (da): adressebetegnelse  
    definition (da): en læsbar struktur, som identificerer adressen fuldstændigt  
    legalSource: https://www.retsinformation.dk/eli/lta/2018/271#P12  
    ""  
  
    adressebetegnelse: String @cost(weight: "1")  
    bygning: String @cost(weight: "1")  
    datafordelerOpdateringstid: DafDateTime @cost(weight: "1")  
    ""  
  
    Grunddatamodel info:  
    type: CharacterString  
    multiplicity: 0..1  
    URI: https://data.gov.dk/model/profile/dar/dørbetegnelse  
    prefLabel (da): dørbetegnelse  
    definition (da): betegnelse, som angiver den adgangsdør e.l. som adressen identificerer  
    legalSource: https://www.retsinformation.dk/eli/lta/2018/271#P22 || https://www.retsinforma  
    ""  
}
```

Hvordan sender man en POST forespørgsel ?

Det er kompliceret at lave en GET - det er bedre at sende et POST request afsted.

- Dette kan gøres med en række forskellige stykker software - Jeg bruger Postman til mine eksempler:

www.postman.com/

Basis request

Endpoint : `graphql.datafordeler.dk/DAR/v1?apiKey=API-KEY`

```
query {  
  DAR_Adresse(  
    registreringstid: "2025-06-04T00:00:00Z"  
    virkningstid: "2025-06-04T00:00:00Z"  
    first: 10  
  ) {  
    nodes {  
      adressebetegnelse  
      id_lokalId  
      husnummer  
      doerbetegnelse  
      etagebetegnelse  
      status  
      registreringFra  
      virkningFra  
    }  
  }  
}
```

Lad os se et par eksempler

Udviklingsleverance 3 - Det der kommer i december

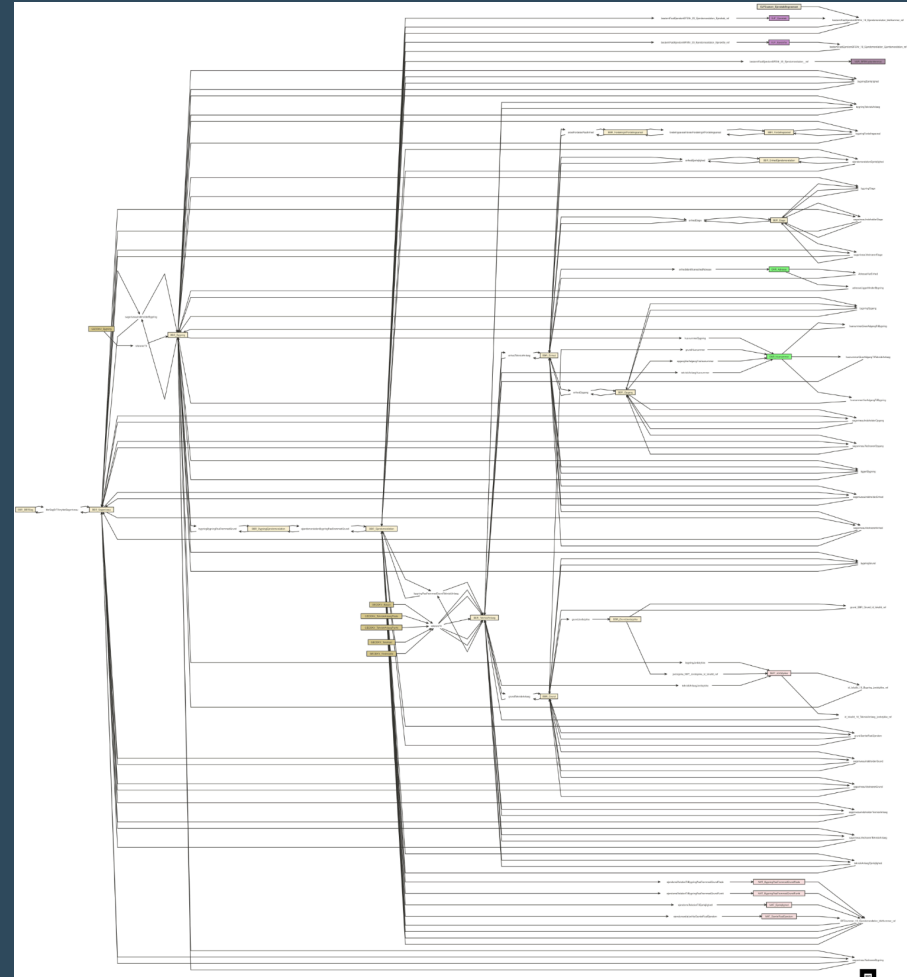
Status lige nu

Delleverance 3 er i test - alle kan tilgå testmiljøerne, hvis man opretter sig...

Forventes at blive frigivet 3. december

Grunddataprogrammet

- Programmet er bygget op med en masse relationer.
- Nu får vi mulighed for at søge via disse relationer i GraphQL.
- For at kunne udnytte dette, skal man dog kende programmet ret godt !



Det Flexible schema

- 75.000 linjers information - det er totalt uoverskueligt. Alle registre samlet i en samlet schemafil, med interne relationer defineret
- KDS laver et stort diagram, vi har internt i Septima forsøgt os med at bryde dem ned i stumper...
(Eksemplerne her ligger på Github)

Hvordan ser en FLEX forespørgsel ud ?

```
query {  
  DAR_Adresse(  
    registreringstid: "2025-06-04T00:00:00Z"  
    virkningstid: "2025-06-04T00:00:00Z"  
    first: 2  
    where: {  
      adressebetegnelse: { eq: "Lærkevej 7, 2600 Glostrup" }  
    }  
  ) {  
    nodes {  
      adressebetegnelse  
      id_lokalId  
      husnummer  
      doerbetegnelse  
      status  
      # Adresse → Husnummer  
      adresseHarHusnummer {  
        id_lokalId  
        husnummertekst  
        postnummer  
        # Husnummer → Adgangspunkt  
        HusnummerHarAdgangspunkt {  
          id_lokalId  
          position {  
            wkt  
            crs  
          }  
        }  
      }  
    }  
  }  
}
```

Og hvad bliver svaret ?

```
{
  "data": {
    "DAR_Adresse": {
      "nodes": [
        {
          "adressebetegnelse": "Lærkevej 7, 2600 Glostrup",
          "id_lokalId": "0a3f50a4-bd23-32b8-e044-0003ba298018",
          "husnummer": "0a3f507c-6354-32b8-e044-0003ba298018",
          "doerbetegnelse": "",
          "status": "3",
          "adresseHarHusnummer": {
            "id_lokalId": "0a3f507c-6354-32b8-e044-0003ba298018",
            "husnummertekst": "7",
            "postnummer": "a26174bd-f602-463c-9a4a-92413ffba23f",
            "HusnummerHarAdgangspunkt": {
              "id_lokalId": "0a3f507c-6354-32b8-e044-0003ba298018",
              "position": {
                "wkt": "POINT (714172.33 6175205.89)",
                "crs": 25832
              }
            }
          }
        }
      ]
    }
  }
}
```

Lad os se et par eksempel mere...

Relevante fokuspunkter

- Hvis man via relationer skal hente mange data, kan man ende med indlejret paging
- Der er mange af de 'smarte' funktioner i GraphQL som er deaktiveret fra KDS på løsningen:
 - Brug af alias'er på kolonner
 - Flere root-nodes
 - Max 20 joins i en query
 - Ingen LIKE operator, og ingen lower/uppercase
- Cost beregningen kan komme i spil
- Versioner på schemaer

Hvornår sker skiftet ?

Finansministeriet har i forbindelse med Forslag til finanslov 2026 bevilget budget til parallelldrift på Datafordeleren frem til den 30. juni 2026. Det betyder, at de ikke-moderniserede tjenester skal være udfaset, og at parallelldrift vil ophøre 30. juni 2026.

<https://datafordeler.dk/vejledning/modernisering/>

Alle ressourcer vi har brugt i dag, kan hentes på Github

[Github.com/MFuglsang/graphql_datafordeleren](https://github.com/MFuglsang/graphql_datafordeleren)

Her er bla. en pdf med en lang række eksempler fra både UL2 og UL3 - langt flere end vi har set i dag...

Tak for i dag

Har i spørgsmål efterfølgende, så hiv gerne fat i mig :
morten.fuglsang@septima.dk